

Homework Assignment- Hybrid Intrusion Detection with Snort and Machine Learning

This assignment extends the ideas from the Snort and AI-for-networks labs into a more advanced hybrid intrusion-detection exercise. Students must create their own traffic, operate Snort in a controlled lab, extract features from their captures, train an anomaly or classification model, and combine both sources of evidence into a single incident prioritisation pipeline.

Learning Objectives

- Deploy and operate Snort in a repeatable IDS workflow.
- Write and test custom detection rules in addition to the default rule set.
- Engineer network or flow features suitable for ML-based anomaly detection or classification.
- Compare signature-based detection, ML-based detection, and a hybrid fusion strategy.
- Analyse false positives, blind spots, and evasion opportunities in a realistic network-monitoring context.

Provided Material

- No mandatory instructor material is provided.
- Students must generate their own benign and malicious traffic or justify clearly any public capture used as a supplement.
- Any external tools, public traces, pretrained models, or scripts must be documented in the report.

Operational / Ethical Constraints

- Run all scans, probes, and floods only against systems or namespaces that you own and control in a closed lab.
- Do not direct any traffic at public hosts, university infrastructure, or third-party services.
- The goal is defensive detection and evaluation, not offensive impact.

Required Tasks

1. Design the Testbed and Traffic Plan

Create a small yet realistic network-monitoring setup and explain how ground truth will be established.

Steps:

- Define a topology containing at minimum an IDS sensor, one target service, and one traffic-generation endpoint.
- Document IP ranges or namespace configuration and state how HOME_NET will be set in Snort.
- Deploy at least one reachable service that can be probed legitimately and maliciously, for example HTTP on a non-standard port.
- Write a short traffic plan that separates benign actions from attack scenarios and explains how each event will be labelled.

Minimum evidence expected:

- A topology diagram or equivalent schematic.
- A short description of how the student will regenerate the lab if the environment is reset.

2. Generate and Label Traffic

Produce a dataset that contains meaningful benign activity and multiple malicious behaviours rather than a trivially separable toy example.

Steps:

- Generate benign traffic such as browsing, normal service requests, periodic pings, or scripted client activity.
- Include at least four malicious categories: a TCP SYN scan, an ICMP-based probe or flood, a TCP connection flood or burst pattern, and an HTTP probe or suspicious request sequence.
- Include at least one stealthier or low-and-slow variation that is less obvious to threshold-based rules.
- Capture the traffic using pcap and maintain a label file, timeline, or event log that records when each scenario starts and ends.
- Ensure that the final dataset contains enough observations for evaluation; as a guide, at least 1,000 labelled flows or windows should be available overall unless a different justification is provided.

Minimum evidence expected:

- A reproducible method for generating benign traffic and each malicious scenario.
- A label source such as a CSV timeline, event notebook, or scenario manifest.

3. Deploy Snort and Engineer Custom Rules

Configure Snort so that it produces useful alerts and verify both default and custom detections.

Steps:

- Run Snort in IDS or logging mode and document the command or configuration used.
- Analyse the default alerts produced by your required attack traffic and identify what is and is not detected.
- Write at least three custom rules in the local rules file.
- One rule must detect an HTTP GET or suspicious HTTP request to your chosen service port.
- One rule must detect ICMP echo requests or a related ICMP pattern originating outside HOME_NET.
- One additional rule must be chosen and justified by the student, for example repeated SYN behaviour, a custom URI pattern, suspicious user-agent text, or a UDP probe signature.
- Use unique student-defined sid values and demonstrate that each custom rule actually fires under the intended condition.

Minimum evidence expected:

- The text of the custom rules.
- Alert evidence showing that the rules triggered as expected.

4. Extract Features and Train an ML Detector

Build a second detector that operates from traffic-derived features rather than signatures alone.

Steps:

- Extract flow-level or window-level features from the captured traffic, such as packets, bytes, duration, TCP flags, protocol, destination-port diversity, connection rate, inter-arrival statistics, or request-level indicators.

- Justify why the chosen features are relevant to the attack categories in your dataset.
- Train either a supervised classifier or an anomaly-detection model; acceptable choices include Random Forest, Gradient Boosting, Isolation Forest, One-Class SVM, or an autoencoder if well justified.
- Use a clean separation between training and testing data and explain how leakage is avoided.
- Report the preprocessing steps, feature scaling or encoding choices, and hyperparameters at a level that another student could reproduce.

Minimum evidence expected:

- A feature description table or concise appendix in the report.
- A trained detector with reproducible training code.

5. Build a Hybrid Fusion and Incident Prioritisation Layer

Combine the Snort evidence and the ML evidence into a single decision or ranking pipeline.

Steps:

- Design a fusion method that combines alert severity, rule confidence, ML score, or anomaly probability into a final incident score.
- Define thresholds or policy logic for low, medium, and high-risk events.
- Compare three configurations: Snort-only, ML-only, and the hybrid detector.
- Explain how your hybrid method handles conflicts, for example a strong anomaly score with no matching rule or a rule hit on otherwise normal traffic.

Minimum evidence expected:

- A documented fusion formula or decision procedure.
- An incidents output file or ranked event list.

6. Evaluate Detection Quality

Measure the quality of the detectors with enough detail to support a credible comparison.

Steps:

- Report precision, recall, F1-score, and false positive rate for the main configurations.
- Include at least one confusion matrix and one compact visual summary such as a metrics table or chart.
- Comment on alert volume or analyst burden, for example how many alerts would require manual review.
- Provide at least three case studies where one detector caught behaviour that the other detector missed or handled poorly.
- Discuss whether any metric is unstable because of class imbalance, low sample counts, or evaluation design choices.

Minimum evidence expected:

- A comparative metrics section backed by logs and labels.
- At least three short case analyses with supporting evidence.

7. Evasion and Reflection Analysis

Analyse how attackers might evade the system and how the design could be improved for a production setting.

Steps:

- Discuss at least three realistic evasion ideas such as rate limiting, jitter, protocol variation, fragmentation, or blending into legitimate activity.

- State clearly which evasions would mostly affect Snort, which would mostly affect the ML detector, and which could affect both.
- Explain how you would reduce false positives in a real deployment without simply hiding suspicious activity.
- Reflect on the limitations of zero-day detection in both signature-based and ML-based approaches.

Minimum evidence expected:

- A reflective section in the report that moves beyond restating the metric tables.

Deliverables

- Source code or scripts for traffic generation, feature extraction, training, and evaluation.
- Snort configuration notes and the custom local rules.
- A report containing the topology, methodology, evaluation, case studies, and reflection.
- Screenshots or captured evidence of Snort alerts for the required scenarios.
- A reproducibility guide explaining how the marker can regenerate the lab and results.
- If large pcap files are omitted because of size limits, provide a script or concise instructions to recreate them and include a small representative sample.

Optional Advanced Tasks

- Implement a low-and-slow attack variant deliberately tuned to evade one detector, then improve the hybrid design and show the effect.
- Compare two ML approaches, such as Isolation Forest versus an autoencoder, and explain which one is more suitable for your lab setting.
- Add a feedback loop that automatically adjusts rule thresholds or analyst triage scores based on measured false positives.
- Repeat the exercise with Suricata or an alternative IDS and compare operational differences while preserving the same traffic plan.